NumPy

Cheat Sheet for Data Scientists

Essential array operations every ML engineer must know

Swipe to explore →

[ARRAY] Array Creation & Basics

Create Arrays

```
import numpy as np
np.array([1, 2, 3])           # 1D array
np.array([[1,2],[3,4]])       # 2D array
np.zeros((3, 4))              # zeros matrix
np.ones((2, 3))               # ones matrix
np.full((2,3), 7)             # filled with 7
```

Special Arrays

```
np.eye(3)              # identity matrix
np.arange(0, 10, 2)    # [0,2,4,6,8]
np.linspace(0,1,5)     # 5 evenly spaced
np.random.rand(3,3)    # uniform [0,1]
np.random.randn(3,3)   # normal dist
np.empty((2,3))        # uninitialised
```

Array Properties

```
arr.shape        # dimensions tuple
arr.ndim         # number of axes
arr.size         # total elements
arr.dtype        # element data type
arr.itemsize     # bytes per element
arr.nbytes       # total bytes
```

Reshape & Flatten

```
arr.reshape(3, 4)       # new shape
arr.flatten()           # 1D copy
arr.ravel()             # 1D view
arr.T                   # transpose
arr.swapaxes(0, 1)      # swap axes
np.expand_dims(a, 0)    # add axis
```

TIP Always check arr.shape and arr.dtype first — shape mismatches are the #1 NumPy bug in ML pipelines.

[INDEX] Indexing & Slicing

Basic Indexing

```
arr[0]            # first element
arr[-1]           # last element
arr[1, 2]         # row 1  col 2
arr[0:3]          # rows 0, 1, 2
arr[:, 1]         # all rows  col 1
```

Advanced Indexing

```
arr[[0, 2, 4]]           # fancy index
arr[arr > 5]             # boolean mask
arr[np.where(arr > 5)]   # where filter
arr[1:4, 2:5]            # 2D slice
np.take(arr, [0,2], axis=0)
```

Boolean & Conditional Selection

```
mask = arr > 10                              # boolean array
arr[mask]                                    # filtered values
arr[(arr > 5) & (arr < 20)]                  # compound condition
np.where(arr > 0, arr, 0)                    # replace negatives with 0
np.select([arr<0, arr==0], [-1, 0], default=1)  # multi-condition select
arr[np.ix_([0,1], [2,3])]                    # 2D fancy (outer) indexing
```

TIP Boolean indexing returns a COPY; slicing returns a VIEW — modifying a view changes the original array!

[MATH] Math & Statistical Operations

Element-wise Math

```
np.add(a, b)          # a + b
np.subtract(a, b)     # a - b
np.multiply(a, b)     # a * b (element)
np.divide(a, b)       # a / b
np.power(a, 2)        # a squared
np.sqrt(arr)          # square root
```

Aggregations (axis aware)

```
np.sum(arr, axis=0)   # column sums
np.mean(arr, axis=1)  # row means
np.median(arr)        # median
np.std(arr)           # std deviation
np.var(arr)           # variance
np.cumsum(arr)        # cumulative sum
```

Rounding & Clipping

```
np.round(arr, 2)      # round to 2 dp
np.floor(arr)         # floor values
np.ceil(arr)          # ceiling values
np.clip(arr, 0, 100)  # clamp to range
np.abs(arr)           # absolute value
np.sign(arr)          # sign: -1 / 0 / +1
```

Sorting & Searching

```
np.sort(arr)              # sorted copy
np.argsort(arr)           # sort indices
np.argmin(arr)            # index of min
np.argmax(arr)            # index of max
np.unique(arr)            # unique values
np.unique(arr, return_counts=True)
```

TIP Always pass axis= to aggregations — omitting it collapses ALL dimensions into a single scalar.

[LINALG] Linear Algebra & Matrix Operations

Matrix Multiplication

```
np.dot(A, B)          # dot product
A @ B                 # matrix multiply
np.matmul(A, B)       # same as @
np.outer(a, b)        # outer product
np.inner(a, b)        # inner product
np.kron(A, B)         # Kronecker product
```

Decomposition & Inverse

```
np.linalg.inv(A)        # inverse
np.linalg.det(A)        # determinant
vals, vecs = np.linalg.eig(A)
U, S, Vt = np.linalg.svd(A)
Q, R = np.linalg.qr(A)  # QR decomp
np.linalg.norm(A)       # matrix norm
```

Stacking & Splitting

```
np.vstack([a, b])          # vertical stack
np.hstack([a, b])          # horizontal stack
np.concatenate([a,b], axis=0)
np.stack([a,b], axis=0)    # new axis stack
np.split(arr, 3)           # split into 3
np.hsplit(arr, 2)          # split columns
```

Broadcasting Rules

```
# Dims align right-to-left
# (3,1) + (1,4)  =>  (3,4)
a = np.array([[1],[2],[3]])    # (3,1)
b = np.array([10,20,30,40])   # (4,)
a + b                          # (3,4)
a[:,np.newaxis]  # add axis
```

TIP Broadcasting avoids loops — reshape with np.newaxis to align dimensions before element-wise ops.

[PERF] Data Types, Memory & Performance

Data Type Control

```
arr.astype(np.float32)       # cast dtype
np.int8, np.int16, np.int32, np.int64
np.float16, np.float32, np.float64
np.bool_, np.complex64
arr.dtype.itemsize            # bytes/element
np.finfo(np.float32).max     # type max
```

Memory & Views

```
arr.flags['OWNDATA']         # is copy?
arr.copy()                   # force full copy
np.shares_memory(a, b)       # shared check
arr.nbytes                   # total bytes
arr.strides                  # byte-step tuple
np.ascontiguousarray(arr)    # C-order
```

Random Generation (New API)

```
rng = np.random.default_rng(42)  # seed
rng.integers(0, 10, size=(3,3))
rng.normal(0, 1, size=100)
rng.uniform(0, 1, size=50)
rng.choice([1,2,3,4], size=5)
rng.shuffle(arr)     # in-place shuffle
```

Vectorisation Tips

```
# AVOID Python loops — use ufuncs
np.vectorize(func)(arr)     # last resort
np.frompyfunc(f,1,1)(arr)   # ufunc wrap
# Prefer @ over np.dot for clarity
# Use axis= instead of manual loops
# float32 halves memory on GPU
```

TIP Use float32 instead of float64 in ML — halves memory usage and significantly speeds up GPU training.

Save this for your next ML / Data Science project!

[BUILD]

Create Efficiently

np.zeros / ones / eye

Avoid Python lists

[SLICE]

Index Smartly

Boolean masks are fast

Views vs copies matter

[CALC]

Math with axis=

Always specify axis=

Broadcasting > loops

Found this useful? Like & share with your network!